

Система логістики інтернет-аптеки

1. Список вимог

Наразі ми з групою розробляємо систему для інтернет-аптеки. Під час оформлення замовлення користувач може обрати різні способи отримання медикаментів: кур'єром до дверей, у відділення пошти або самовивіз із найближчої фізичної аптеки. Оскільки логіка розрахунку вартості, маршрутизації та навіть перевірки кошика (наприклад, рецептурні препарати не можна відправляти звичайною поштою) для кожного способу суттєво відрізняється, нам необхідно відокремити логіку створення способу доставки від самого процесу оформлення замовлення (checkout).

Список вимог:

1. Система повинна підтримувати різні типи доставки: CourierDelivery (кур'єр), PostalDelivery (пошта) та PharmacyPickup (самовивіз з аптеки).
2. Кожен тип доставки має власну унікальну бізнес-логіку для розрахунку вартості та валідації замовлення.
3. Клієнтський код (модуль обробки кошика) не повинен бути жорстко прив'язаний до конкретних класів доставки, а має працювати з ними виключно через загальний інтерфейс Delivery.
4. Система має бути гнучкою: якщо в майбутньому ми додамо доставку за допомогою дронів (через терміновість ліків) або спеціалізовану доставку з температурним контролем, це вимагатиме лише створення нового класу та його фабрики, без втручання в існуючий код кошика.
5. Для інстанціювання об'єктів доставки повинен використовуватися патерн "Фабричний метод".

2. Похідний код (Java)

Цей код демонструє роботу "Factory Method" в контексті аптеки.

```
// 1. Спільний інтерфейс для всіх типів доставки
public interface Delivery {
    double calculateCost();
    void arrangeDelivery(String orderDetails);
}

// 2. Конкретні продукти (Способи доставки)
public class CourierDelivery implements Delivery {
    @Override
    public double calculateCost() {
        return 150.0; // Вартість послуг кур'єра
    }

    @Override
    public void arrangeDelivery(String orderDetails) {
        System.out.println("Формування доставки КУР'ЄРОМ за адресою пацієнта для: "
+ orderDetails);
    }
}

public class PostalDelivery implements Delivery {
    @Override
    public double calculateCost() {
        return 80.0; // Вартість поштової відправки
    }

    @Override
    public void arrangeDelivery(String orderDetails) {
        System.out.println("Передача пакунку з ліками до поштового відділення
для: " + orderDetails);
    }
}

public class PharmacyPickup implements Delivery {
    @Override
    public double calculateCost() {
        return 0.0; // Самовивіз безкоштовний
    }

    @Override
    public void arrangeDelivery(String orderDetails) {
        System.out.println("Бронювання препаратів на касі в обраній аптеці для: "
+ orderDetails);
    }
}

// 3. Базовий клас Фабрики
public abstract class DeliveryFactory {
    // Фабричний метод
    public abstract Delivery createDelivery();
}
```

```

// Загальна бізнес-логіка оформлення замовлення
public void processOrder(String orderDetails) {
    Delivery delivery = createDelivery();
    double cost = delivery.calculateCost();
    System.out.println("Вартість доставки складе: " + cost + " грн.");
    delivery.arrangeDelivery(orderDetails);
}
}

// 4. Конкретні фабрики для створення відповідних типів доставки
public class CourierFactory extends DeliveryFactory {
    @Override
    public Delivery createDelivery() {
        return new CourierDelivery();
    }
}

public class PostalFactory extends DeliveryFactory {
    @Override
    public Delivery createDelivery() {
        return new PostalDelivery();
    }
}

public class PickupFactory extends DeliveryFactory {
    @Override
    public Delivery createDelivery() {
        return new PharmacyPickup();
    }
}

// 5. Клієнтський код (Наприклад, контролер кошика інтернет-аптеки)
public class PharmacyApp {
    public static void main(String[] args) {
        String orderInfo = "Замовлення #777 (Вітамін С, Ібупрофен)";

        // Користувач обирає самовивіз
        System.out.println("--- Клієнт обрав самовивіз ---");
        DeliveryFactory pickupFactory = new PickupFactory();
        pickupFactory.processOrder(orderInfo);

        // Інший користувач замовляє ліки кур'єром
        System.out.println("\n--- Клієнт обрав кур'єра ---");
        DeliveryFactory courierFactory = new CourierFactory();
        courierFactory.processOrder(orderInfo);
    }
}

```

Лістинг 1 – вихідний код

3. UML Діаграма класів

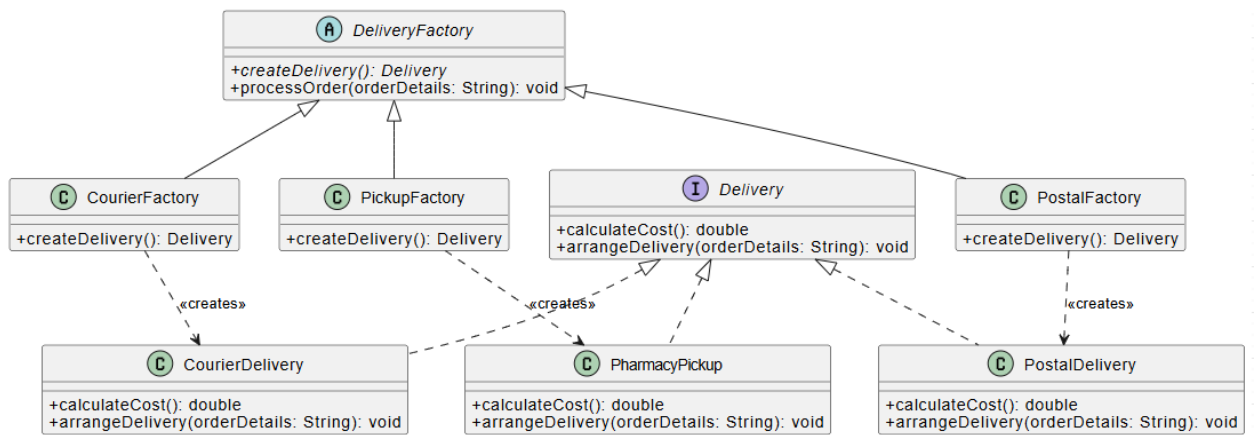


Рисунок 1 – побудована UML діаграма